REACT.JS
MASTER NOTES • PART 1

Foundations, Vite, Components & Props

BSc Computer Science | Full-Stack Development

A structured study guide with explanations, code examples, diagrams and revision checklists.

Contents

- 1. Introduction to React
- 2. React Overview and JSX
- 3. Import and Export
- 4. Real DOM vs Virtual DOM
- 5. Setting Up React with Vite
- 6. Components Explained
- 7. Understanding Props
- 8. Final Revision Sheet and Checklist

How to use these notes
Read the concept first, then type the code into your own Vite project. Change values and experiment instead of only copying examples. Code snippets are kept as selectable text for easy copying.

1. Introduction to React

1.1 What is React?

React is an open-source JavaScript library used to build interactive user interfaces (UIs), especially for web applications. It lets developers create reusable UI components and efficiently update the interface when application data changes.

React was developed at Facebook (now Meta) and is maintained by Meta and the open-source community.

1.2 Why do we need React?

In a traditional website, JavaScript may manually find HTML elements and update them when something changes. This works for small pages, but can become difficult to manage as the application grows.

```
<h2 id="count">0</h2>
<button onclick="increase()">Increase</button>

<script>
  let count = 0;

  function increase() {
    count++;
    document.getElementById("count").textContent = count;
  }
</script>
```

React takes a declarative approach: describe what the UI should look like for the current data, and React manages the necessary updates.

1.3 Important features of React

- Component-based architecture: break a UI into smaller, reusable parts.
- Declarative programming: describe the desired UI instead of manually specifying every DOM update.
- Reusable code: reuse a component in multiple places.
- Efficient updates: React calculates the changes needed to bring the DOM in line with rendered output.
- One-way data flow: data commonly flows from parent components to child components through props.
- JSX: syntax that lets you write HTML-like markup inside JavaScript.
- Ecosystem: can be combined with routing, state-management and other tools.

1.4 What can you build with React?

Application type	Examples
Dashboards	Analytics, charts and admin panels
E-commerce	Product listings, carts and filters
Social applications	Feeds, comments and profiles
Productivity apps	Task managers, planners and boards

1.5 Is React a framework?

React is generally classified as a JavaScript library, not a complete framework.

React	A full-stack framework
Focuses primarily on the UI layer	Can provide a broader application structure
Additional libraries may be chosen for features	Often includes routing and other conventions
Can be used with Vite for development	Examples include Next.js and React Router framework mode

Remember
React does not replace JavaScript, HTML or CSS. It uses JavaScript to describe UI, JSX for markup, and CSS or other styling solutions for appearance.

2. React Overview

2.1 How React works

A simplified React application follows this cycle. Not every render results in a DOM change, and not every render is caused by a state change.

A simplified rendering cycle

Data or state changes

↓

React renders the component

↓

React reconciles the new output with the previous output

↓

React commits required changes to the DOM

↓

The browser displays the updated UI

2.2 What is JSX?

JSX (JavaScript XML) is a JavaScript syntax extension that allows you to write HTML-like markup inside JavaScript code. JSX is transformed into JavaScript instructions that React can use.

```
function App() {
  return <h1>Hello, React!</h1>;
}

export default App;
```

2.3 JSX rules to remember

- Return one parent element, or use a fragment (<code><>...</></code>), when returning multiple sibling elements.
- Close all elements, including elements such as <code><img /></code> and <code><input /></code>.
- Use <code>className</code> instead of <code>class</code> for CSS classes.
- Use curly braces <code>{}</code> to insert JavaScript expressions into JSX.
- Use camelCase for most JSX event and property names, such as <code>onClick</code>.

```
function App() {
  const name = "Abrar";

  return (
    <>
      <h1>Hello, {name}!</h1>
      <button onClick={() => alert("Welcome!")}>
        Click Me
      </button>
    </>
  );
}

export default App;
```

Here, {name} inserts the variable into the heading, while onClick defines a click handler.

2.4 Declarative programming

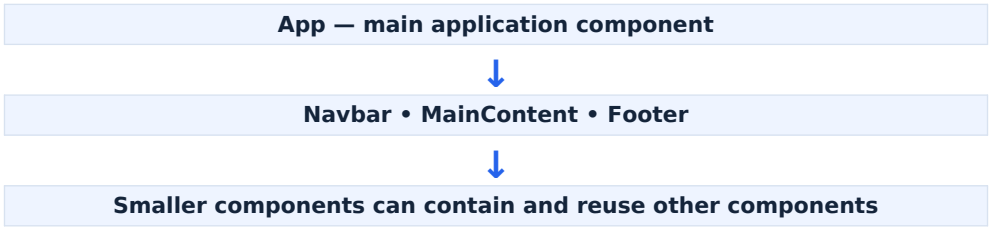
In declarative programming, you describe what result you want rather than writing every step needed to produce it.

```
function Greeting() {
  const name = "Abrar";
  return <h1>Hello, {name}</h1>;
}
```

React handles the necessary DOM updates when the rendered output needs to change.

2.5 Component-based architecture

A page can be composed from smaller components



An e-commerce website might contain Navbar, ProductList, ProductCard, Cart and Footer components.

3. Import and Export in React

3.1 What are import and export?

Export makes a variable, function or component available for use in another JavaScript module. Import allows another module to access that exported value.

React applications are often divided into multiple files so code stays organized, reusable and easier to maintain.

3.2 Default export

A default export is the main value a module exports. A module can have only one default export.

```
// Navbar.jsx
export default function Navbar() {
  return <h2>My Website</h2>;
}
```

```
// App.jsx
import Navbar from "./Navbar";

function App() {
  return (
    <>
      <Navbar />
      <h1>Welcome</h1>
    </>
  );
}

export default App;
```

The imported default value can be given a different local name, although keeping the original name is usually clearer.

```
import SiteNavbar from "./Navbar";
```

3.3 Named export

A named export exports a value under a specific name. A module can have multiple named exports.

```
// components.jsx
export function Navbar() {
  return <h2>Navbar</h2>;
}

export function Footer() {
  return <p>Footer section</p>;
}
```

```
// App.jsx
import { Navbar, Footer } from "./components";

function App() {
  return (
    <>
      <Navbar />
      <Footer />
    </>
  );
}

export default App;
```

Named imports use curly braces. You can rename a named import with the keyword as.

```
import { Navbar as SiteNavbar } from "./components";
```

3.4 Default export vs named export

Feature	Default export	Named export
Number per module	One	Multiple
Import syntax	import Navbar from "./Navbar"	import { Navbar } from "./Navbar"
Curly braces	Not required	Required
Import name	Can be chosen freely	Must match unless aliased
Typical use	Main component of a file	Multiple utilities or components

3.5 Combining default and named exports

```
export function Footer() {
  return <footer>Copyright 2026</footer>;
}

export default function App() {
  return <Footer />;
}

// Import both:
import App, { Footer } from "./App";
```

3.6 File paths in imports

```
import Navbar from "./Navbar";
import Footer from "./components/Footer";
import logo from "./assets/logo.png";
```

- <code>./</code> means the current directory.
- <code>../</code> means the parent directory.
- Check spelling, capitalization, directory names and the file's actual location.

3.7 Common mistakes

- Using curly braces for a default export: use <code>import Navbar from './Navbar'</code> when Navbar is the default export.
- Forgetting to export the component you want to import.
- Using an incorrect file path or filename.
- Confusing the default export name with a named export.

4. Real DOM vs Virtual DOM

4.1 What is the DOM?

DOM stands for Document Object Model. It is a programming interface that represents an HTML document as a tree of objects. JavaScript can use it to access and modify elements, attributes, text and other document properties.

```
<html>
  <body>
    <h1>Hello</h1>
    <p>Welcome to my website</p>
  </body>
</html>
```

Simplified DOM tree




```
document.querySelector("h1").textContent = "Hello, Abrar!";
```

This changes the heading's text in the actual DOM.

4.2 What is the Virtual DOM?

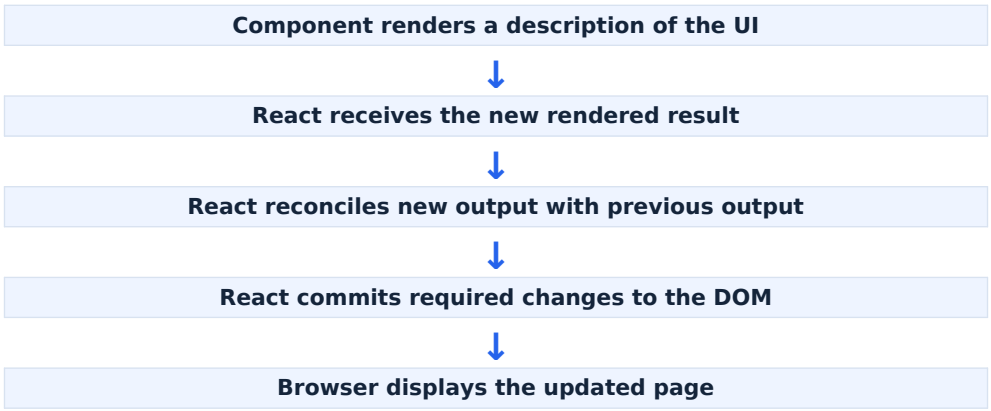
Virtual DOM is a commonly used term for React's in-memory representation of the UI. React uses rendering and reconciliation to determine which changes need to be applied to the actual DOM.

```
function App() {
  return <h1>Hello, Abrar!</h1>;
}
```

When a component's output changes, React compares the new rendered result with the previous one and determines the necessary updates. The Virtual DOM is a conceptual simplification, not a separate browser DOM or a complete description of every React internal.

4.3 Simplified update process

How React updates rendered output



4.4 Real DOM vs Virtual DOM

Feature	Real DOM	Virtual DOM concept
Meaning	Browser's actual document object tree	In-memory representation of rendered UI
Location	Maintained by the browser	Managed by React
Purpose	Represents the actual web document	Helps React calculate UI changes
Browser-visible?	Yes	Not as a separate browser DOM
Updating	Can be modified through DOM APIs	React reconciles output and commits required changes
Responsibility	Browser DOM APIs and rendering system	React's rendering and reconciliation system

4.5 React counter example

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <h2>Count: {count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increase
      </button>
    </div>
  );
}

export default Counter;
```

- <code>setCount</code> requests a state update.
- React renders the component with the new count.
- React reconciles the new output with the previous output.
- React updates the relevant DOM content when necessary.

4.6 Is the Virtual DOM always faster?

No. It is inaccurate to say the Virtual DOM is always faster than direct DOM manipulation. React's main advantage is a predictable, declarative way to manage complex UI updates. Performance depends on the application, component design, amount of work and browser rendering costs.

Exam-ready conclusion
The Real DOM represents the actual web document in the browser. The Virtual DOM is a useful conceptual description of React's in-memory UI representation. React reconciles changes in rendered output and commits the necessary updates to the Real DOM.

5. Setting Up React with Vite

5.1 What is Vite?

Vite is a modern frontend build tool that provides a development server and builds an application for production. It is commonly used to set up React projects because it offers a fast development workflow and sensible defaults.

- Starts a local development server.
- Transforms JSX and other supported source files for development.
- Supports Hot Module Replacement (HMR), so many code changes appear without a full page reload.
- Builds and optimizes application assets for production.

React vs Vite
React is the UI library. Vite is the development and build tool. They serve different purposes.

5.2 Prerequisites

- Node.js — runs JavaScript tools outside the browser.
- npm — package manager included with Node.js.
- VS Code or another code editor.
- A modern web browser.

```
node -v
npm -v
```

Both commands should display version numbers. Vite has Node.js version requirements; check its official guide if your version is outdated.

5.3 Create a React project

```
npm create vite@latest my-react-app -- --template react
cd my-react-app
npm install
npm run dev
```

- <code>npm create vite@latest</code> runs the Vite scaffolding tool.
- <code>my-react-app</code> is the project folder name.
- <code>--template react</code> selects the JavaScript React template.
- <code>npm install</code> installs the project's dependencies.
- <code>npm run dev</code> starts the development server.

Vite commonly prints a local address such as http://localhost:5173/. Open the exact URL shown in your terminal.

Already have a Vite project?
You do not need to create another one. Open its folder in the terminal and run npm run dev.

5.4 Typical project structure

```
my-react-app/
├── node_modules/
├── public/
├── src/
│   ├── App.jsx
│   ├── main.jsx
│   ├── index.css
│   └── App.css
├── index.html
└── package.json
```

File/folder	Purpose
node_modules/	Contains installed packages and dependencies; generally not committed to Git.
public/	Static assets served as-is.
src/App.jsx	Common starting point for the main UI component.
src/main.jsx	Entry point that creates the React root and renders App.
src/index.css	Common location for global styles.
src/App.css	Common location for app-specific styles.
index.html	Main HTML document, usually containing the root element.
package.json	Project metadata, dependency declarations and scripts.

5.5 Understanding index.html

```
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

The root element is where React mounts the application. The module script loads the React entry-point file.

5.6 Understanding main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

- <code>createRoot</code> creates a React root for the DOM element.
- <code>document.getElementById('root')</code> finds the HTML element.
- <code>.render(...)</code> tells React what to display.
- <code><App /></code> is the main application component.
- <code><StrictMode></code> enables additional development checks; it is not a visible HTML element.

5.7 Understanding App.jsx

```
function App() {
  return (
    <div>
      <h1>My First React App</h1>
      <p>React is working!</p>
    </div>
  );
}

export default App;
```

5.8 Understanding package.json scripts

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  }
}
```

- <code>npm run dev</code> starts development.
- <code>npm run build</code> generates a production build.
- <code>npm run preview</code> locally previews the production build.

5.9 How the app runs

From HTML to the browser UI

index.html — provides the root element

↓

main.jsx — creates the React root and renders App

↓

App.jsx — defines the main UI and uses other components

↓

Browser — displays the rendered application

5.10 Common setup problems

- node/npm not recognized: install Node.js and reopen the terminal.
- Missing packages: confirm the project directory and run <code>npm install</code>.
- Changes not appearing: save files, check terminal and browser console, and verify the correct file.
- Port in use: Vite may choose another port; open the URL it prints.
- Blank page: check the console for JSX errors, missing imports, incorrect paths or errors in main.jsx.

6. Components Explained

6.1 Definition of a component

A React component is a reusable, independent piece of a user interface that can contain markup, logic and styling. A component might represent a button, navigation bar, product card or entire page.

6.2 Create your first component

```
function Welcome() {
  return <h1>Welcome to React!</h1>;
}

export default Welcome;
```

- <code>function Welcome()</code> defines a JavaScript function named Welcome.
- The capital W follows React's component naming convention.
- <code>return</code> specifies the JSX output.
- <code>export default</code> makes the component available to other modules.

6.3 Use a component

```
function Welcome() {
  return <h2>Welcome!</h2>;
}

function App() {
  return (
    <div>
      <h1>My Website</h1>
      <Welcome />
      <Welcome />
    </div>
  );
}

export default App;
```

The Welcome component is used twice, so its output appears twice. Define a component once and reuse it wherever needed.

6.4 Component naming rules

Component names must begin with an uppercase letter. Lowercase JSX names such as <code><nav></code> refer to built-in HTML elements.

```
// Correct
function Navbar() {
  return <nav>Navigation</nav>;
}

// Use it as:
<Navbar />
```

Rule
Component names start with uppercase letters; built-in HTML tag names are lowercase.

6.5 Components must return valid JSX

A component can return a single JSX element or a fragment containing multiple sibling elements.

```
// Correct: a fragment groups sibling elements
function App() {
  return (
    <>
      <h1>Hello</h1>
      <p>Welcome</p>
    </>
  );
}
```

A div can also be used when a real wrapper element is appropriate. Fragments group elements without adding an extra DOM element.

6.6 Component composition and nesting

Component composition means building larger interfaces by combining smaller components.

```
function Navbar() {
  return <nav>My Website</nav>;
}

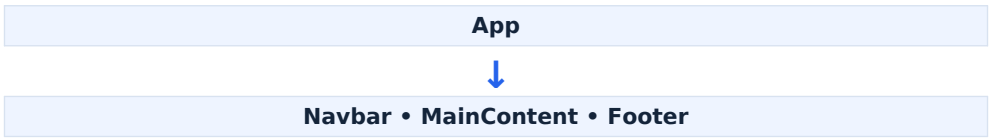
function MainContent() {
  return <main>Welcome to my website!</main>;
}

function Footer() {
  return <footer>Copyright 2026</footer>;
}

function App() {
  return (
    <>
      <Navbar />
      <MainContent />
      <Footer />
    </>
  );
}

export default App;
```

Component composition



You can update the navigation bar in one place rather than editing every page individually.

6.7 Split components into separate files

```
src/
├── components/
│   ├── Navbar.jsx
│   └── Footer.jsx
├── App.jsx
└── main.jsx
```

```
// components/Navbar.jsx
export default function Navbar() {
  return <nav>My Website</nav>;
}
```

```
// App.jsx
import Navbar from "./components/Navbar";
import Footer from "./components/Footer";

function App() {
  return (
    <>
      <Navbar />
      <h1>Home Page</h1>
      <Footer />
    </>
  );
}

export default App;
```

This combines reusable components with imports and exports.

6.8 Important component rules

- A component should normally be a pure function of its inputs: the same props and other relevant inputs should produce the same JSX output.
- Do not directly modify props.
- Keep component definitions at the top level rather than defining components inside other components.
- Use meaningful names, such as ProductCard, LoginForm and Navbar.
- Extract repeated UI into reusable components when that improves clarity.
- Use state when a component needs to remember changing information; state is covered later.

7. Understanding Props

7.1 What are props?

Props (short for properties) are read-only inputs passed from one React component to another, usually from a parent component to a child component. They let components receive information and display different content without rewriting the component for every situation.

7.2 Why do we need props?

```
// Without props: fixed content
function ProjectCard() {
  return (
    <div>
      <h2>Snake Game</h2>
      <p>A game built using JavaScript.</p>
    </div>
  );
}
```

This component always displays the same information. Props let you customize it.

```
function ProjectCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.description}</p>
    </div>
  );
}
```

7.3 Passing props to a component

```
function ProjectCard(props) {
  return (
    <div>
      <h2>{props.name}</h2>
      <p>{props.description}</p>
    </div>
  );
}

function App() {
  return (
    <>
      <ProjectCard
        name="Snake Game"
        description="A game built using JavaScript."
      />

      <ProjectCard
        name="Photo Editor"
        description="An image editor built with vanilla JavaScript."
      />

      <ProjectCard
        name="Kanban Board"
        description="A task management application."
      />
    </>
  );
}

export default App;
```

- App is the parent component.
- ProjectCard is the child component.

- name and description are props passed by the parent.
- The child receives the values through its props parameter.
- {props.name} and {props.description} display the values in JSX.

7.4 Props using destructuring

JavaScript destructuring extracts values from an object. Instead of repeatedly writing props.name, you can extract name directly.

```
// Without destructuring
function ProjectCard(props) {
  return <h2>{props.name}</h2>;
}

// With destructuring
function ProjectCard({ name }) {
  return <h2>{name}</h2>;
}
```

```
function Student({ name, age, course }) {
  return (
    <div>
      <h2>{name}</h2>
      <p>Age: {age}</p>
      <p>Course: {course}</p>
    </div>
  );
}

function App() {
  return (
    <Student
      name="Abrar"
      age={21}
      course="BSc Computer Science"
    />
  );
}

export default App;
```

Remember
Destructuring is a JavaScript feature, not a special React feature.

7.5 Passing different data types

A. String props

```
<Student name="Abrar" />
```

String values can be written inside quotation marks.

B. Number props

```
<Student age={21} />
```

Curly braces pass the number 21 rather than the string "21".

C. Boolean props

```
<Student isEnrolled={true} />

// Shorthand for true:
<Student isEnrolled />
```

D. Array props

```
function Skills({ skills }) {
  return <p>{skills.join(", ")}</p>;
}

function App() {
  return <Skills skills={["HTML", "CSS", "JavaScript"]} />;
}

export default App;
```

E. Object props

```
function Profile({ user }) {
  return (
    <div>
      <h2>{user.name}</h2>
      <p>{user.course}</p>
    </div>
  );
}

function App() {
  const student = {
    name: "Abrar",
    course: "BSc CS",
  };

  return <Profile user={student} />;
}

export default App;
```

F. Function props

```
function Child({ onGreet }) {
  return <button onClick={onGreet}>Greet</button>;
}

function App() {
  function greet() {
    alert("Hello!");
  }

  return <Child onGreet={greet} />;
}

export default App;
```

A function prop lets a child trigger logic defined by its parent. Pass greet, not greet(), when you want to pass the function itself rather than call it immediately.

7.6 Default values for props

A default value provides a fallback when a prop is missing or undefined.

```
function Greeting({ name = "Guest" }) {
  return <h2>Welcome, {name}!</h2>;
}

// Usage:
<Greeting name="Abrar" />
<Greeting />
```

The outputs are “Welcome, Abrar!” and “Welcome, Guest!” respectively. The default is used for undefined, not automatically for null or an empty string.

7.7 Props are read-only

A child component should not directly modify the props it receives. Assigning a new value to a local parameter does not update the parent's prop; mutating an object received through props should also be avoided.

```
// Avoid changing a prop
function Student({ name }) {
  return <h2>{name}</h2>;
}

function App() {
  return <Student name="Abrar" />;
}
```

If information needs to change, the parent can pass updated props or a component can manage its own changing information with state.

7.8 Props vs state

Feature	Props	State
Purpose	Inputs passed into a component	Information that can change over time
Ownership	Supplied by parent/caller	Managed by the component that owns it
Direct modification	Do not modify in receiving component	Update with the appropriate state setter
Change mechanism	Parent supplies new props	State setter requests an update
Typical use	Product name, price, user profile	Counter, menu visibility, form input

For now
Props are inputs. State is a component's managed, changing information. You'll study state in a later topic.

7.9 The children prop

React provides a special prop called children. It contains the content placed between a component's opening and closing tags.

```
function Card({ children }) {
  return (
    <div className="card">
      {children}
    </div>
  );
}

function App() {
  return (
    <Card>
      <h2>My Project</h2>
      <p>Built using React.</p>
    </Card>
  );
}

export default App;
```

This is useful for reusable cards, layouts, dialogs and wrapper components.

7.10 Parent-to-child data flow

One-way data flow



A child can communicate with a parent indirectly by calling a function passed down as a prop. This does not mean the child directly modifies the parent's props.

8. Final Revision Sheet

Topic	Key takeaway
React	JavaScript library for building user interfaces
JSX	HTML-like syntax written inside JavaScript
Declarative UI	Describe the desired UI; React manages necessary updates
DOM	Browser's object representation of the HTML document
Virtual DOM	Conceptual term for React's in-memory UI representation
Vite	Development server and build tool
main.jsx	Creates the React root and renders the app
App.jsx	Common starting point for the main UI component
Component	Reusable piece of UI
Component naming	Start component names with a capital letter
export	Makes values available to other modules
import	Brings exported values into a module
Default export	One per module; imported without curly braces
Named export	Multiple per module; imported using curly braces
Props	Read-only inputs passed into components
Destructuring	Extracts values from an object
children	Content placed between a component's tags
One-way data flow	Props commonly flow from parent to child

Official documentation

- React — Quick Start: <link href="https://react.dev/learn">https://react.dev/learn</link>
- React — Importing and Exporting Components: <link href="https://react.dev/learn/importing-and-exporting-components">react.dev/learn/importing-and-exporting-components</link>
- React — Passing Props to a Component: <link href="https://react.dev/learn/passing-props-to-a-component">react.dev/learn/passing-props-to-a-component</link>
- Vite — Getting Started: <link href="https://vite.dev/guide/">vite.dev/guide</link>

Checklist: Can you do these?

- Explain what React is and why it is used.
- Explain JSX and component-based architecture.
- Distinguish the DOM from React's in-memory UI representation.
- Create and run a React project with Vite.
- Explain the roles of index.html, main.jsx and App.jsx.
- Create components and reuse them in other components.
- Use default and named imports and exports correctly.
- Pass strings, numbers, objects, arrays and functions through props.
- Use destructuring and default prop values.
- Explain why props should not be modified by the receiving component.

Practical exercise
Build a small React page with a Navbar, Student component and three ProjectCard components. Put components in separate files, use imports and exports, and pass different props to each card. This tests nearly every concept in Part 1.